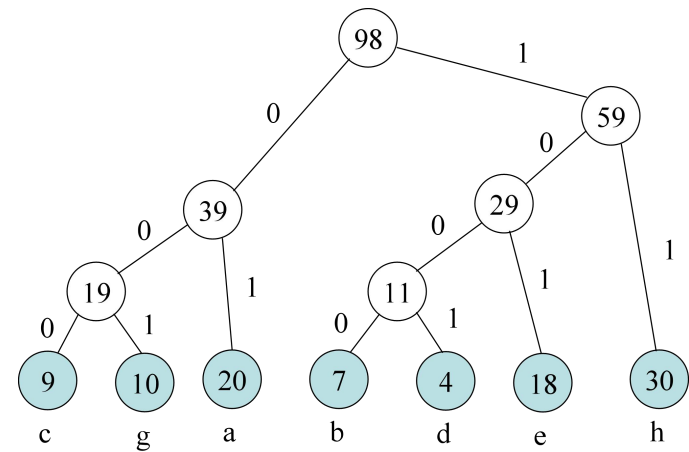


课程名称： 算法设计与分析 学分： 2 教学大纲编号： 06023102 试卷编号： 考试方式： 满分分值： 100 考试时间： 120 分钟 组卷日期： 组卷教师(签字)： 审定人(签字)：	所以： $T(n) = 2^k \sum_{j=1}^k j 2^{-j} = 2 \cdot 2^k - 2 + k = 2n - 2 + \log_2^n = \Theta(n)$ （3分） (2) 解： 理想情形下，规模为 n 的原始数组被分割为 2 个规模为 n/2 的子数组。因此，时间复杂度可以递归表示如下： $T(n) = \begin{cases} \Theta(1) & \text{if } n=1 \\ 2T(n/2) + \Theta(n) & \text{if } n>1 \end{cases}, \quad (2.5 \text{ 分})$ 解之得： $T(n) = \Theta(n \log n) \quad (2.5 \text{ 分})$ 最坏情形下，规模为 n 的原始数组被分割为 1 个规模为 n-1 的子数组。因此，时间复杂度可以递归表示如下： $T(n) = \begin{cases} \Theta(1) & \text{if } n=1 \\ T(n-1) + \Theta(n) & \text{if } n>1 \end{cases} \quad (2.5 \text{ 分})$ 解之得： $T(n) = \Theta(n^2) \quad (2.5 \text{ 分})$
一、填空题（每空格 1 分，共 20 分） 1. 输入 输出 确定性 有限性 确定性 2. a<=b a>=b a=b a<b 3. T(n) = aT(n/b) + s(n) 4. 分治 解决冗余 子问题的解 5. 2ⁿ 3ⁿ 6. 定长的向量形式 深度优先 7. 较高 8. 深度优先 广度优先 二、计算题(每题 10 分，共 20 分) (1) 解： $T(n) = \sum_{i=0}^{k-1} (k-i)2^i \text{ 令 } k-i=j$ 那么， $T(n) = \sum_{j=1}^k j 2^{k-j} = 2^k \sum_{j=1}^k j 2^{-j}$ （2分） 又因为： $\sum_{j=1}^k j 2^{-j} = 1 \cdot \frac{1}{2} + 2 \cdot \frac{1}{2^2} + 3 \cdot \frac{1}{2^3} + \dots + k \cdot \frac{1}{2^k}$ $= (1 \cdot \frac{1}{2} + 1 \cdot \frac{1}{2^2} + 1 \cdot \frac{1}{2^3} + \dots + 1 \cdot \frac{1}{2^k}) + (1 \cdot \frac{1}{2^2} + 1 \cdot \frac{1}{2^3} + \dots + 1 \cdot \frac{1}{2^k})$ $+ (1 \cdot \frac{1}{2^3} + \dots + 1 \cdot \frac{1}{2^k}) + \dots + 1 \cdot \frac{1}{2^k}$ $= (1 - \frac{1}{2^k}) + (\frac{1}{2^1} - \frac{1}{2^k}) + (\frac{1}{2^2} - \frac{1}{2^k}) + (\frac{1}{2^3} - \frac{1}{2^k}) + \dots + (\frac{1}{2^{k-1}} - \frac{1}{2^k})$ $= (\frac{1}{2^0} + \frac{1}{2^1} + \frac{1}{2^2} + \dots + \frac{1}{2^{k-1}}) + k \cdot \frac{1}{2^k}$ $= 2(1 - \frac{1}{2^k}) + k \cdot \frac{1}{2^k} \quad (5 \text{ 分})$ 三、解答题(共 60 分) (1) 解： 输入：整数数组 A[1...n]，整数 x 输出：x 在数组 A[1...n]中出现的频度 1. Frequence(A,1,n,x) 过程 Frequence (A,low,high,x) - if high – low = 0 - if A[low] = x return 1 - else return 0; - end if - if high – low > 0 - mid ← ⌊(low+high)/2⌋ - return Frequence(A,low,mid,x)+Frequence(A,mid+1,high,x) - end if (8 分) $T(n) = \begin{cases} \Theta(1) & , n = 1 n=2 \\ T(n/2) + T(n/2) & , n > 2 \end{cases} \quad (4 \text{ 分})$ $T(n) = \Theta(n) \quad (3 \text{ 分})$	

(2) 解：

哈夫曼编码树如下图所示：



(10 分)

每个字符的编码为：

a: 01; b: 1000; c: 000; d: 1001; e: 101; g: 001; h: 11
(5 分)

(3) 解：

输入：已经排序(非升序)的整型数组 $A[1 \cdots n]$ ，整数 x

输出：若 A 中存在两个整数的和恰好为 x ，返回 true；否则返回 false

1. return sumX(1,n)

Procedure: sumX(s,t)

1. if $s = t$ or $s > t$
2. return false;
3. else if $A[s] + A[t] = x$
4. return true;
5. else if $A[s] + A[t] > x$
6. return sumX(s+1,t);
7. else
8. return sumX(s,t-1);
9. end if

(12 分)

时间复杂度为：

$$T(n) = 1 + T(n-1) = O(n) \quad (3 \text{ 分})$$

(4) 解：

设 $T(i,j)$ 表示从格子(1,1)走到格子(i,j)所能取得的苹果总数。很显然， $T(N,M)$ 就是问题的解。

(a) 据题意， $T(i,j)$ 可以递归定义如下：

$$T(i,j) = \begin{cases} C(1,1) & \text{if } i=j=1 \\ T(i,j-1)+C(i,j) & \text{if } i=1, j>1 \\ T(i-1,j)+C(i,j) & \text{if } j=1, i>1 \\ \max(T(i,j-1), T(i-1,j))+C(i,j) & \text{if } i>1, j>1 \end{cases} \quad (5 \text{ 分})$$

(b)

输入：给定的 $N \times M$ 的矩阵 $C_{N \times M}$

输出：所能取得的最大桔子数目

1. $T(1,1) \leftarrow C(1,1)$
2. for $j \leftarrow 2$ to M
3. $T(1,j) = T(1,j-1) + C(1,j)$
4. end
5. for $i \leftarrow 2$ to N
6. $T(i,1) = T(i-1,1) + C(i,1)$
7. end
8. for $i \leftarrow 2$ to N
9. for $j \leftarrow 2$ to M
10. $T(i,j) \leftarrow \max(T(i,j-1), T(i-1,j)) + C(i,j)$
11. end for
12. end for
13. result $\leftarrow T(N,M)$
14. return result;

(7 分)

时间复杂度为 $O(NM)$ (3 分)